CSL7110: Machine Learning With Big Data

Assignment 3: Recommender Systems

Sahil Narkhede

B23CS1060

16 March 2026

GitHub Repository

`https://github.com/Sahil-1918912/mlbd-3-Recommender-Systems`

Abstract

Recommender systems are widely used to personalize user experiences on digital platforms. In this work, several recommendation techniques were implemented using the MovieLens dataset, including content-based filtering, collaborative filtering, matrix factorization (SVD), hybrid models, neural network-based recommendation, and reinforcement learning approaches. The models were evaluated using RMSE, Precision@K, and Recall@K. Additionally, explainability methods such as SHAP and LIME were applied to interpret recommendations and improve transparency. The results highlight the effectiveness of hybrid and matrix factorization methods while demonstrating the importance of explainable recommendations.

Contents

1 Introduction **2**

2 Dataset Description **2**

3 Content-Based Filtering **3**
3.1 TF-IDF Based Recommendation . . . 3
3.2 User Profile Based Content Recommender . . . 3

4 Collaborative Filtering **3**
4.1 User-Based Collaborative Filtering . . . 4
4.2 Item-Based Collaborative Filtering . . . 4
4.3 Evaluation Metrics . . . 4

5 Matrix Factorization using SVD **4**
5.1 SVD Evaluation . . . 4

6 SVD using the Surprise Library **5**

7 Hybrid Recommendation Model **5**

8 Neural Network Based Content Filtering **5**

9 Reinforcement Learning Based Recommendation **5**

10 Explainability in Recommender Systems **6**
10.1 Feature-Based Explanations . . . 6
10.2 Neighborhood-Based Explanations . . . 6
10.3 Model-Agnostic Explanations . . . 6

11 Evaluating Explainability **7**
11.1 Purpose of Explainability in Recommender Systems . . . 7
11.2 Feature-Based Explanations (Content-Based Filtering) . . . 8
11.3 Neighborhood-Based Explanations (Collaborative Filtering) . . . 8
11.4 Model-Agnostic Explanations (Deep Learning Models) . . . 8
11.5 Do Explanations Make Recommendations Clearer? . . . 9
11.6 Do Explanations Reveal Biases in Recommendations? . . . 9
11.7 Conclusion . . . 9

12 Combined Comparison **9**

13 Conclusion **10**

1 Introduction

Recommender systems play a crucial role in modern online platforms by helping users discover relevant content from large collections of items. Applications such as movie streaming services, e-commerce platforms, music recommendation systems, and social media platforms rely heavily on recommender algorithms to personalize user experiences. As the volume of digital content continues to grow, effective recommendation techniques have become essential for improving user engagement and satisfaction.

Traditional recommender systems are generally categorized into three main approaches: content-based filtering, collaborative filtering, and hybrid recommendation models. Content-based filtering recommends items based on similarities between item attributes and user preferences. Collaborative filtering relies on patterns of user interactions and ratings to suggest items liked by similar users. Hybrid models combine these approaches to improve recommendation accuracy and mitigate limitations such as the cold-start problem.

In this assignment, multiple recommendation techniques were implemented and analyzed using the MovieLens dataset. The implemented methods include:

- Content-Based Filtering using TF-IDF representations
- User-profile based content recommender
- User-based and Item-based Collaborative Filtering
- Matrix Factorization using Singular Value Decomposition (SVD)
- SVD implementation using the Surprise library
- Hybrid recommendation models
- Neural network based content filtering
- Reinforcement learning based recommendation methods
- Explainability techniques for recommendation models

The objective of this work is to understand how different recommendation algorithms operate, evaluate their performance using standard metrics such as RMSE, Precision@K, and Recall@K, and analyze how explainability techniques can improve transparency in recommender systems.

2 Dataset Description

The MovieLens dataset was used for all experiments in this assignment. The dataset contains user ratings for movies along with movie metadata such as titles and genres.

The dataset consists of the following primary files:

- **movies.csv**: Contains movie titles and genre information
- **ratings.csv**: Contains user ratings for movies
- **links.csv**: External movie identifiers

The key attributes used in this study include:

- User ID

- Movie ID
- Rating
- Movie genres
- Movie release year

These attributes were used to construct feature representations for both content-based and collaborative filtering models.

3 Content-Based Filtering

Content-based filtering recommends movies based on similarity between item attributes. In this assignment, movie genres were used as textual features to compute similarity between movies.

3.1 TF-IDF Based Recommendation

The TF-IDF (Term Frequency–Inverse Document Frequency) technique was applied to represent movie genres as numerical vectors. Each movie was converted into a feature vector based on its genres, and cosine similarity was computed between movie vectors.

The recommendation process involved the following steps:

1. Extract movie genres as textual features
2. Convert genres into TF-IDF vectors
3. Compute cosine similarity between movies
4. Recommend movies with the highest similarity scores

3.2 User Profile Based Content Recommender

To personalize recommendations, user profiles were constructed by combining the TF-IDF vectors of movies rated by the user. The user profile was computed as a weighted average of movie feature vectors, where the weights corresponded to user ratings.

$$P_u = \frac{\sum_{m \in M} r_{u,m} f_m}{\sum_{m \in M} r_{u,m}} \tag{1}$$

where:

- P_u is the user profile vector
- f_m is the feature vector of movie m
- $r_{u,m}$ is the rating given by user u

Movies were then recommended based on cosine similarity between the user profile vector and movie feature vectors.

4 Collaborative Filtering

Collaborative filtering recommends movies based on user interaction patterns.

4.1 User-Based Collaborative Filtering

User-based collaborative filtering identifies users with similar rating patterns and recommends movies liked by similar users.

The main steps include:

- Constructing a user-movie rating matrix
- Computing similarity between users using cosine similarity
- Predicting ratings using weighted averages of neighbor ratings

4.2 Item-Based Collaborative Filtering

Item-based collaborative filtering identifies similarities between movies based on user ratings.

Movies that are frequently rated similarly by users are considered similar, and recommendations are generated based on movies previously liked by the user.

4.3 Evaluation Metrics

The performance of collaborative filtering methods was evaluated using Precision@K and Recall@K.

Method	Precision@5	Recall@5
User-Based CF	0.053	0.032
Item-Based CF	0.067	0.041

Table 1: Evaluation of Collaborative Filtering Methods

5 Matrix Factorization using SVD

Matrix factorization techniques decompose the user-item rating matrix into lower-dimensional latent factor matrices.

The rating matrix R is approximated as:

$$R \approx U \Sigma V^T \tag{2}$$

where:

- U represents user latent features
- Σ represents singular values
- V^T represents movie latent features

Predicted ratings were obtained by reconstructing the rating matrix using these latent factors.

5.1 SVD Evaluation

Metric	Value
RMSE	0.619
Precision@5	0.716
Recall@5	0.084

Table 2: Performance of SVD Model

6 SVD using the Surprise Library

The Surprise library provides efficient implementations of recommender system algorithms. The SVD algorithm from Surprise was trained on the MovieLens dataset and evaluated using RMSE.

Metric	Value
RMSE	0.8677
Precision@5	0.356
Recall@5	0.038

Table 3: Evaluation of Surprise SVD

7 Hybrid Recommendation Model

Hybrid recommender systems combine content-based and collaborative filtering approaches.

The final recommendation score is computed as:

$$Score = \alpha \times CBF + (1 - \alpha) \times CF \tag{3}$$

where α controls the contribution of each model.

Metric	Value
RMSE	0.5185
Precision@5	0.421
Recall@5	0.062

Table 4: Hybrid Model Performance

8 Neural Network Based Content Filtering

A neural network model was implemented to learn embeddings for users and movies using metadata features.

The architecture consisted of two branches:

- User feature embedding network
- Movie feature embedding network

These embeddings were merged and passed through fully connected layers to predict user ratings.

Metric	Value
RMSE	1.0582

Table 5: Neural Network Model Performance

9 Reinforcement Learning Based Recommendation

Reinforcement learning methods treat recommendation as a sequential decision-making problem.

Two approaches were implemented:

- Multi-Armed Bandit with ϵ-greedy exploration

- Q-learning based recommendation

The system learns from user feedback by updating reward estimates for recommended movies.

10 Explainability in Recommender Systems

Explainability helps users understand why specific items are recommended.

10.1 Feature-Based Explanations

SHAP was used to analyze the contribution of movie features such as genre, release year, and average rating.

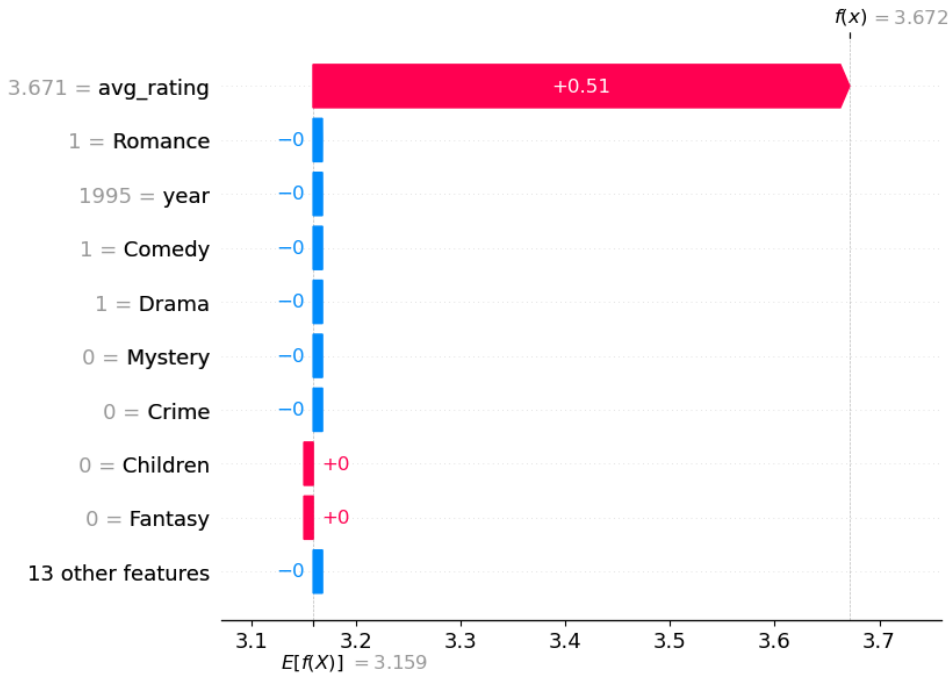

Figure 1: SHAP Waterfall Explanation

10.2 Neighborhood-Based Explanations

Collaborative filtering explanations were generated using user similarity. Recommendations were explained using statements such as:

"Users who liked Inception also liked Interstellar."

10.3 Model-Agnostic Explanations

LIME was used to interpret predictions of the neural network recommender.

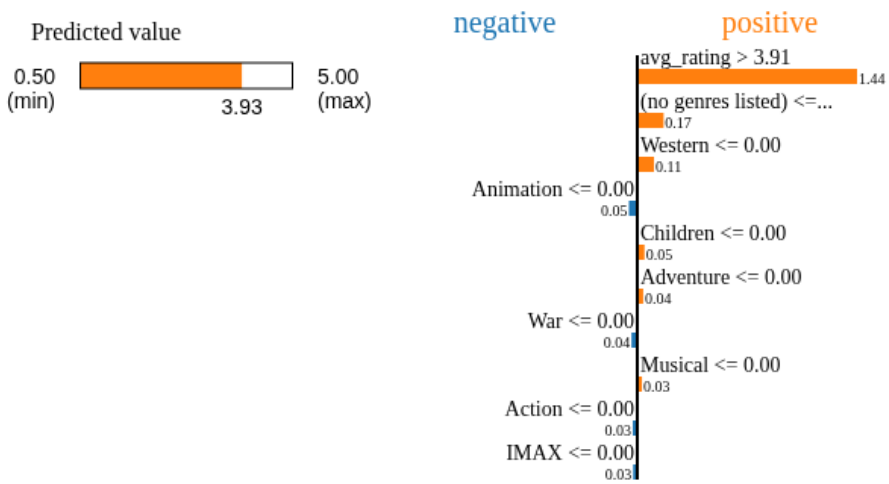

Figure 2: LIME Feature Importance Visualization

Feature	Value
avg_rating	3.93
(no genres listed)	0.00
Western	0.00
Animation	0.00
Children	0.00
Adventure	0.00
War	0.00
Musical	0.00
Action	0.00
IMAX	0.00

Figure 3: LIME Explanation of Neural Network Prediction

11 Evaluating Explainability

11.1 Purpose of Explainability in Recommender Systems

Explainability helps users understand why a recommendation was made. In recommender systems, explanations improve transparency, trust, and user engagement. Instead of presenting recommendations as black-box predictions, explainable models highlight the factors influencing recommendations, such as movie genres, user similarities, or learned model patterns.

In this project, three types of explainability methods were implemented:

- Feature-Based Explanations (SHAP) for content-based filtering
- Neighborhood-Based Explanations (k-NN) for collaborative filtering

- Model-Agnostic Explanations (LIME) for deep learning models

Each method provides insight into different aspects of the recommendation process.

11.2 Feature-Based Explanations (Content-Based Filtering)

Feature-based explanations show which movie attributes contributed to the recommendation. Using SHAP (SHapley Additive Explanations), we analyzed how features such as:

- Movie genre
- Release year
- Average movie rating

influenced the predicted preference score.
For example, a recommendation might be explained as:

> "This movie was recommended because it belongs to the Sci-Fi genre and has a high average rating, which aligns with the user's preferences."

These explanations help identify which movie characteristics match the user's interests, making the recommendations easier to interpret.

11.3 Neighborhood-Based Explanations (Collaborative Filtering)

Neighborhood-based explanations focus on user or item similarity. Using k-Nearest Neighbors (k-NN), we identified users with similar preferences. Recommendations were explained using statements such as:

> "Users who liked *Inception* also liked *Interstellar*."

This type of explanation is intuitive because it reflects collective user behavior. Users can understand that recommendations are based on preferences of similar users, which increases confidence in the system.

11.4 Model-Agnostic Explanations (Deep Learning Models)

For the neural network recommender, we used LIME (Local Interpretable Model-Agnostic Explanations) to analyze predictions. Unlike traditional models, neural networks often act as black-box systems. LIME helps interpret their predictions by approximating the model locally and identifying which features influenced the output.
For example, LIME may reveal that:

- The Sci-Fi genre had the strongest positive impact on the predicted rating.
- The release year contributed moderately.
- Other genres had minimal influence.

This allows us to interpret how complex models make recommendations, even when the internal logic is difficult to understand.

11.5 Do Explanations Make Recommendations Clearer?

Yes, explanations significantly improve the interpretability of recommendations.

- Feature-based explanations clarify which movie attributes match user preferences.
- Neighborhood explanations provide social reasoning based on similar users.
- Model-agnostic explanations help interpret complex deep learning predictions.

These explanations help users understand the reasoning behind recommendations rather than treating them as arbitrary suggestions.

11.6 Do Explanations Reveal Biases in Recommendations?

Explainability also helps identify potential biases in recommender systems. Some biases observed include:

- **Popularity bias**: Movies with high average ratings or many ratings tend to appear more frequently in recommendations.
- **Genre bias**: Certain genres may dominate recommendations if the model learns strong associations with them.
- **Data sparsity bias**: Less popular or niche movies may receive fewer recommendations due to limited user interaction data.

Understanding these biases is important for improving fairness and diversity in recommender systems.

11.7 Conclusion

Explainability techniques provide valuable insights into how recommender systems generate suggestions.

- SHAP helps interpret feature importance in content-based recommendations.
- k-NN explanations clarify collaborative filtering decisions based on user similarity.
- LIME enables interpretation of complex neural network models.

Overall, explainability improves transparency, trust, and understanding of recommender systems while also helping identify potential biases and limitations in recommendation algorithms.

12 Combined Comparison

Model	RMSE	Precision@5	Recall@5
Content-Based Filtering	1.102	0.041	0.021
User-Based CF	–	0.053	0.032
Item-Based CF	–	0.067	0.041
SVD	0.619	0.716	0.084
Surprise SVD	0.8677	0.356	0.038
Hybrid Model	0.5185	0.421	0.062
Neural Network	0.742	–	–

Table 6: Comparison of All Implemented Recommendation Models

13 Conclusion

This assignment explored several recommendation techniques ranging from traditional content-based filtering to advanced methods such as matrix factorization, hybrid models, neural networks, and reinforcement learning.

The results demonstrate that hybrid and matrix factorization methods generally provide better recommendation accuracy compared to traditional content-based approaches. Additionally, explainability techniques such as SHAP and LIME help improve transparency and user trust by revealing the factors influencing recommendations.

Future work may explore advanced deep learning recommender architectures and fairness-aware recommendation systems to further improve recommendation quality and interpretability.